Camunda 8 SaaS + Spring Boot — Student Leave Workflow Lab

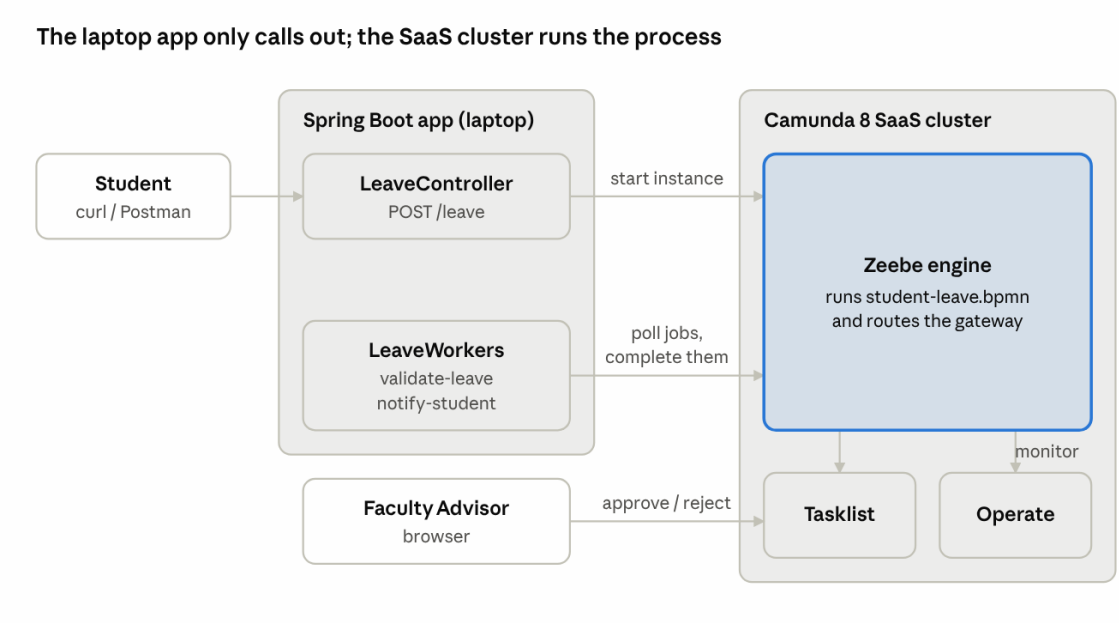
What students build

Students build a **Student Leave Application** process that runs on a Camunda 8 SaaS cluster, with all the business logic in a Spring Boot app on their laptop. It covers the four things every Camunda 8 + Spring project needs: deploying a model from code, starting an instance over REST, running service tasks as job workers, and a human task in Tasklist.

The process logic:

1. A student submits a leave request through a REST call (`POST /leave`).
2. **Validate leave request** (Java worker) computes the number of days and checks the dates are not in the past.
3. A gateway routes the request: invalid dates are rejected, up to 2 days is auto-approved, more than 2 days goes to the **Faculty Advisor approval** form in Tasklist.
4. **Notify student** (Java worker) logs the final decision (stand-in for an e-mail).

The app talks to the cluster outbound only, so it works from a campus network or home Wi-Fi with no port forwarding. The cluster never calls the laptop; workers poll it for jobs.



lab architecture · laptop app and SaaS cluster

Every arrow from the laptop points outward: the controller starts instances and the workers fetch jobs, so no inbound port is needed.

Step-by-step lab sheet (the whole lab on one page)

Students follow these 16 steps in order; Parts A–D below explain each one.

In Camunda Cloud (browser)

1. Log in to console.camunda.io.
2. Clusters → Create new cluster → name `leave-demo`, nearest region, 8.8 → Create. Wait for **Healthy**.
3. Open the cluster → **API** tab → Create new client → name `leave-app`, tick the Zeebe / Orchestration Cluster scope.
4. Copy Client ID, Client Secret, Cluster ID and Region (Spring Boot tab). Save them safely.

5. Open **Modeler** → New project `Leave Workflow` → Upload `student-leave.bpmn` and `fa-approval.form` (or draw them, Part B).
6. Click the process canvas and confirm Process ID = `student-leave`; click each service task and confirm the task types `validate-leave` and `notify-student`.

On the laptop (IDE)

1. Unzip `leave-workflow.zip` and open it in IntelliJ / VS Code as a Maven project.
2. Paste the four values from step 4 into `src/main/resources/application.yaml`.
3. Read `LeaveWorkers.java`: match each `@JobWorker(type=…)` to a service task.
4. Read `LeaveController.java`: note `bpmnProcessId("student-leave")`.
5. Run `mvn spring-boot:run`. Check the log for the deployment and "started" lines.

Test (browser + terminal)

1. Operate → Processes: confirm *Student Leave Application* is deployed.
2. Send a 2-day request with curl/Postman → instance completes → log shows APPROVED.
3. Send a 5-day request → Operate shows the token waiting at *Faculty Advisor approval*.
4. Tasklist → open the task → Assign to me → tick Approve, add remark → Complete.
5. Operate → open the instance → show the path taken and the Variables panel. Take screenshots for the record.

Project files: `leave-workflow.zip` (shared alongside this guide) contains the full Maven project, the BPMN model and the form.

Prerequisites

Item	Version / note
Camunda account	Free trial or university/team org at console.camunda.io
JDK	17 or later (`java -version`)
Maven	3.9+ (`mvn -v`), or use the IDE's bundled Maven
IDE	IntelliJ IDEA Community or VS Code with Java extensions
HTTP client	Postman, or `curl` in a terminal

The project pins Spring Boot 3.5.6 with `camunda-spring-boot-starter` 8.8.15. This starter replaced the older Spring Zeebe SDK from Camunda 8.8 onward, so older tutorials using `spring-boot-starter-camunda-sdk` or `ZeebeClient` are out of date. Set `camunda.version` in `pom.xml` to the same minor version as your cluster (shown in Console).

Part A — Camunda Console: cluster and API client

The goal of Part A is four values: **Client ID, Client Secret, Cluster ID and Region**. Everything else in the app is fixed.

1. Sign in at console.camunda.io.
2. Open **Clusters → Create new cluster**. Name it (e.g. `leave-demo`), pick the nearest region and the 8.8 generation, then create. Wait until the status shows **Healthy** (a few minutes).
3. Open the cluster and go to the **API** tab → **Create new client**.
4. Name the client `leave-app` and tick the **Zeebe / Orchestration Cluster** scope. Create it.
5. A dialog shows the credentials once. Switch to its **Spring Boot** tab: it prints a ready-made `camunda.client` block. Copy it, or download the credentials file.
6. Keep the secret private: never commit it to GitHub. Each group should create its own client.

If the secret is lost, delete the client and create a new one; it cannot be shown again.

Part B — Web Modeler: the process and the form

Students can either import the two files from the zip (fast) or draw them (better for learning). The IDs below must match the Java code exactly.

Option 1 — import: In Modeler, create a project `Leave Workflow`, then **Upload files** and pick `student-leave.bpmn` and `fa-approval.form` from `src/main/resources/bpmn/`.

Option 2 — draw it yourself:

1. Create a BPMN diagram. Click empty canvas and set **ID** = `student-leave`, **Name** = Student Leave Application.
2. Start event "Leave requested" → **Service task** "Validate leave request". Under **Task definition**, set **Type** = `validate-leave`.
3. Add an **exclusive gateway** with three outgoing flows and these **Condition expressions**:
    - to the merge gateway: `=valid and leaveDays <= 2`
    - to the user task: `=valid and leaveDays > 2`
    - to the merge gateway: `=not(valid)`
4. **User task** "Faculty Advisor approval". Choose **Camunda user task**, then **Form → Link to form** and pick the form from step 7 (form ID `fa-approval-form`).
5. A second exclusive gateway merges the three paths → **Service task** "Notify student" with **Type** = `notify-student` → End event.
6. Check the **Problems** panel is empty.
7. Create a **Form** with ID `fa-approval-form`: a Text view showing `{{studentName}}`, `{{regNo}}`, dates and reason; a **Checkbox** with key `approved`; a **Text area** with key `remarks`.

In the model	Value	Must match in code
Process ID	`student-leave`	`bpmnProcessId("student-leave")` in `LeaveController`
Task type	`validate-leave`	`@JobWorker(type = "validate-leave")`
Task type	`notify-student`	`@JobWorker(type = "notify-student")`
Form ID	`fa-approval-form`	file name in `@Deployment`
Form field keys	`approved`, `remarks`	`@Variable` names in `notifyStudent`

No need to click **Deploy** in Modeler: the Spring Boot app deploys both files on start-up. If students drew their own model, they export it (**Download**) into `src/main/resources/bpmn/` with the same file names.

Spring Boot project from scratch (for the student question)

For their own question, students create a new Spring Boot project instead of unzipping one. It takes about 10 minutes.

1. Open **start.spring.io** (Spring Initializr). Choose: Project **Maven**, Language **Java**, Spring Boot **3.5.x**, Java **17**.
2. Metadata: Group `edu.srmist`, Artifact `outpass-workflow`, Package name `edu.srmist.outpass`, Packaging **Jar**.
3. Dependencies → **Add** → **Spring Web**. Click **Generate**; unzip the download.
4. Open the folder in IntelliJ (File → Open → pick `pom.xml` → Open as Project) or VS Code. Wait for Maven to finish downloading.
5. In `pom.xml`, inside `<dependencies>`, add the Camunda starter (same block as Step C1, version 8.8.x). Reload Maven (IntelliJ: the floating **M** icon).
6. Rename `src/main/resources/application.properties` to `application.yaml` and paste the SaaS block from Step C2 with the group's own four values.
7. Create folder `src/main/resources/bpmn/`. Export the BPMN and the form from Web Modeler (**Download**) into it.
8. In the main class (the one with `@SpringBootApplication`), add `@Deployment(resources = {"classpath:bpmn/<file>.bpmn", "classpath:bpmn/<file>.form"})`.
9. Create package `worker` with a `@Component` class; write one `@JobWorker(type = "…")` method per service task.

10. Create package `api` with a `@RestController` that injects `CamundaClient` and starts the process (copy the pattern from Step C5, change the process ID).
11. Run with `mvn spring-boot:run` or the green ▶ button on the main class; test with curl/Postman, then check Operate and Tasklist.

Package layout students should end with:

```
edu.srmist.outpass
├─ OutpassWorkflowApplication.java   @SpringBootApplication + @Deployment
├─ api/OutpassController.java        POST /outpass
└─ worker/OutpassWorkers.java        check-outpass, issue-outpass
```

Part C — The Spring Boot project

Unzip `leave-workflow.zip` and open the folder in the IDE as a Maven project. Students touch only one file (`application.yaml`); the rest is code to read and explain.

```
leave-workflow/
├─ pom.xml
└─ src/main/
   ├─ java/edu/srmist/leave/
   │  ├─ LeaveWorkflowApplication.java   (@Deployment)
   │  ├─ api/LeaveController.java        (POST /leave starts an instance)
   │  └─ worker/LeaveWorkers.java        (two @JobWorker methods)
   └─ resources/
      ├─ application.yaml                 (SaaS credentials)
      └─ bpmn/student-leave.bpmn, fa-approval.form
```

Step C1 — dependency (`pom.xml`). One starter brings the Camunda client, job workers and auto-deployment:

```
<dependency>
  <groupId>io.camunda</groupId>
  <artifactId>camunda-spring-boot-starter</artifactId>
  <version>${camunda.version}</version>
</dependency>
```

Step C2 — connect to SaaS (`application.yaml`). Paste the four values from Part A. In `saas` mode the client builds the cluster URLs itself, so no address is set by hand.

```
camunda:
  client:
    mode: saas
    auth:
      client-id: <client id>
      client-secret: <client secret>
    cloud:
      cluster-id: <cluster id>
      region: <region, e.g. bru-2>
```

Each value can also come from an environment variable (`CAMUNDA_CLIENT_ID`, `CAMUNDA_CLIENT_SECRET`, `CAMUNDA_CLUSTER_ID`, `CAMUNDA_CLUSTER_REGION`), which keeps secrets out of Git.

Step C3 — deploy from code (`LeaveWorkflowApplication`).

```
@SpringBootApplication
@Deployment(resources = {
        "classpath:bpmn/student-leave.bpmn",
        "classpath:bpmn/fa-approval.form"})
public class LeaveWorkflowApplication { ... }
```

Step C4 — job workers (`LeaveWorkers`). The `type` links a method to a service task. `@Variable` pulls process variables in; the returned `Map` is written back as variables. The job completes automatically when the method returns.

```
@JobWorker(type = "validate-leave")
public Map<String, Object> validateLeave(@Variable String regNo,
                                         @Variable String fromDate,
                                         @Variable String toDate) {
    LocalDate from = LocalDate.parse(fromDate);
    LocalDate to = LocalDate.parse(toDate);
```

```
    long days = ChronoUnit.DAYS.between(from, to) + 1;
    boolean valid = days > 0 && !from.isBefore(LocalDate.now());
    boolean autoApproved = valid && days <= 2;
    return Map.of("leaveDays", days, "valid", valid, "approved", autoApproved);
}

@JobWorker(type = "notify-student")
public Map<String, Object> notifyStudent(@Variable String studentName,
        @Variable String regNo, @Variable Boolean approved, @Variable String remarks) {
    String status = Boolean.TRUE.equals(approved) ? "APPROVED" : "REJECTED";
    log.info("Mail to {} ({}): your leave is {}. Remarks: {}", studentName, regNo, status, remarks);
    return Map.of("leaveStatus", status);
}
```

Step C5 — REST trigger (`LeaveController`). The injected `CamundaClient` starts one process instance per request; the JSON body becomes the process variables.

```
@PostMapping("/leave")
public Map<String, Object> applyLeave(@RequestBody LeaveRequest request) {
    ProcessInstanceEvent instance = camundaClient.newCreateInstanceCommand()
            .bpmnProcessId("student-leave")
            .latestVersion()
            .variables(request)
            .send().join();
    return Map.of("processInstanceKey", instance.getProcessInstanceKey());
}
```

Part D — Run it and watch it in Operate and Tasklist

1. **Start the app:** `mvn spring-boot:run` (or run `LeaveWorkflowApplication` from the IDE). The log should show the deployment of `student-leave` and the two workers opening.
2. **Check the deployment:** Console → cluster → **Operate**. The *Student Leave Application* process now appears in the Processes list.
3. **Short leave (auto-approved)** — dates 2 days apart, in the future:

```
curl -X POST http://localhost:8080/leave -H "Content-Type: application/json" -d
'{"studentName":"Priya","regNo":"RA2311003010001","fromDate":"2026-10-20","toDate":"2026-10-
21","reason":"Medical appointment"}'
```

The response returns a `processInstanceKey`. In Operate the instance is already **Completed**, and the app log prints `Mail to Priya ... APPROVED`.

1. **Long leave (needs FA)** — change `toDate` to `2026-10-24` (5 days) and send again. In Operate the token waits at *Faculty Advisor approval*.
2. **Approve in Tasklist:** Console → **Tasklist** → open the task → **Assign to me** → tick *Approve this leave*, type a remark → **Complete task**. The instance finishes and the log prints the decision with the remark.
3. **Invalid dates** — send a `fromDate` in the past. The gateway takes the *invalid dates* path and the student is told REJECTED without FA involvement.
4. In Operate, open each instance and show students the **Variables** panel (`leaveDays`, `valid`, `approved`, `remarks`) and the highlighted path taken through the diagram.

Postman users: method POST, URL `http://localhost:8080/leave`, Body → raw → JSON, same body as above.

Sample question for students: Hostel Out-Pass Request

SRM hostel students need an out-pass to leave campus. Automate the request with Camunda 8 SaaS and a Spring Boot app, using the leave example as your reference.

Process rules

1. The student submits a request through `POST /outpass` with `studentName`, `regNo`, `hostelBlock`, `outDate`, `returnDate`, `parentPhone`, `purpose`.
2. A service task **Check request** (type `check-outpass`) computes `nights` and sets `valid = false` if the return date is before the out date.

3. A gateway: invalid → reject; `nights = 0` (day out) → auto-approve; `nights >= 1` → **Warden approval** user task with a form (`approved` checkbox, `wardenRemarks`).
4. A service task **Issue out-pass** (type `issue-outpass`) creates a pass number `OP-<regNo>-<outDate>` for approved requests and logs "SMS to parent" with the decision.
5. The process ends with the variable `passStatus` = ISSUED or REJECTED.

Deliverables

- BPMN file and form (exported from Web Modeler), plus the Spring Boot project zip.
- Screenshots: Operate showing one auto-approved, one warden-approved and one rejected instance; Tasklist with the warden form.
- A half-page note: which variables each worker reads and writes.

Marking scheme (10)

Component	Marks
BPMN model correct: IDs, task types, gateway conditions	3
Job workers implemented and connected to SaaS	3
REST endpoint starts the process	1
Warden form linked and working in Tasklist	1
Three test cases shown in Operate + variable note	2

Extension (bonus 2): add a timer boundary event so that if the warden does not act in 24 hours the request is escalated to the Chief Warden.

Solution key (faculty only): `outpass-workflow-solution.zip` holds a working answer: `hostel-outpass.bpmn`, `warden-approval.form`, the two workers and `POST /outpass`. Gateway conditions: `=valid and nights = 0`, `=valid and nights >= 1`, `=not(valid)`.

Worker	Reads	Writes
`check-outpass`	`regNo`, `outDate`, `returnDate`	`nights`, `valid`, `approved` (true for a day out)
`issue-outpass`	`regNo`, `outDate`, `studentName`, `parentPhone`, `approved`, `wardenRemarks`	`passStatus`, `passNumber`

Troubleshooting and exercises

Symptom	Likely cause and fix
`401 Unauthorized` / token error at start-up	Wrong client ID or secret, or the client lacks the Zeebe / Orchestration Cluster scope. Recreate the client.
`UNAVAILABLE` / cannot connect	Wrong `cluster-id` or `region`, or `grpc-address` / `rest-address` set by hand. In SaaS mode remove the address lines.
Instance stuck at a service task	Worker `type` does not match the task type in the model (check spelling and case).
Incident in Operate: "Text cannot be parsed"	Date not in `yyyy-MM-dd` format. Fix the variable in Operate, then **Retry**. A good live demo of incidents.

`Expected to find process ... student-leave`	Process ID in the model is not `student-leave`, or the app has not deployed yet.
User task has no form	Form ID not linked, or the `.form` file missing from `@Deployment`.
Old tutorial code does not compile	It uses `ZeebeClient` / `io.camunda.zeebe.spring.*`. Use `CamundaClient` and `io.camunda.client.annotation.*`.

Exercises for students (build on the same project):

- Add a **timer boundary event** on the FA task: escalate to the HOD after 2 days.
- Replace the log line in `notify-student` with a real e-mail via the **SendGrid** or **REST** connector, so the model, not Java, sends it.
- Add a leave-balance check: a worker that reads a JSON file or a small H2 table and throws a **BPMN error** when the balance is exhausted.
- Write a `GET /leave/{key}` endpoint that returns the instance status.
- Make each group adapt the pattern to their own course project (feedback analysis, OD approval, research-grant request), keeping at least one REST or script integration.